

The Two Research Toolkits — CCRT and CSRTB

A guide to the sibling customization layers that carry one research methodology across Anthropic’s two agent platforms: what lives in each, why they diverge where they do, and which specialists and skills to reach for.

How to read this document. Start here and read straight through — it is built as a teaching arc, not a lookup table. The first four sections explain the *structure*: what the two toolkits are, why one platform difference shapes almost every design decision, and the three-tier model that tells you what is shared and what cannot be. Section 5 is the payoff — the **collaboration working-sets**, the recovered rationale for *why* specific specialists and skills are designed to compose. Section 6 shows you how to act on all of it. The two roster tables (Section 4) and the side-by-side comparison (Section 7) are reference: skim them, then return when you need a specific item. Every per-item line is a faithful one-line gloss of that item’s own description — condensed for a human skimmer, never embellished. This is the human-readable twin of the machine reference `TOOLKIT_ROSTER_CCRT_vs_CSRTB.machine.md`; where that file is written terse for an LLM to parse, this one is written to make a person *understand*.

1. Why this matters to you

You have opened a folder of two toolkits that look almost identical — the same specialist names, the same skill names, a shared methodology — and your first instinct will be the wrong one. You will see `machine.md` in both and assume the two copies are interchangeable; they are not (their bodies are only 65% similar). You will see an agent present in one toolkit and absent from the other and read it as a gap to fix; it usually is not. You will try to copy a working feature from one side to the other verbatim, and you will drag a call that only exists on one platform into a file on the other, breaking it.

The single idea that prevents all three mistakes is this: **these are not two copies of one toolkit — they are one methodology re-expressed for two platforms that differ in exactly one load-bearing way**. Learn that one difference and the three-tier model built on top of it, and the whole roster stops looking like a confusing near-duplicate and starts looking like what it is: a deliberate port, with every similarity and every divergence accounted for.

2. What the two toolkits are

Both toolkits equip a research assistant with the same underlying craft — planning and delegation, scientific modeling, statistics, code review, document authoring, literature management. They differ in which Anthropic product they customize.

CCRT — the Claude Research Toolkit — customizes Claude Code. Claude Code runs as a local process on the user’s own machine. Its customizations install into the global `~/.claude/` directory through an `install.sh` script, and they come in three forms: **agents** (specialist personas, each a Markdown file with a system-prompt body), **skills** (reusable methodology and technique docs an agent can load), and **hooks** (small bash or Python programs that fire automatically on real events — when a turn stops, when a notification arrives, before a tool call runs — and that can *block* an action by exiting with code 2). Counted from the trees on 7 August 2026, CCRT carries **18 agents and 47 skills**.

CSRTB — the Claude Science customization bundle — customizes Claude Science (the platform this document was authored on). Claude Science runs in a remote sandbox. It has no local shell for the user, no local audio device, and — the fact that matters most — **no hook surface**: nothing fires automatically on a turn boundary. What it has instead is the **host.* SDK**: `host.delegate` for launching sub-agents, `host.query` and `host.artifacts` and `host.lineage` for reading a metadata database of everything the project has produced, `host.mcp` for calling connected tools. It also has a **background reviewer** that reads each turn *after* it happens and scores it. Its customizations are **profiles** (the Science analog of an agent) and skills, distributed as a self-installing JSON bundle. Counted the same day, CSRTB carries **18 profiles and 52 skills**.

That is the whole cast. The two sides now match in size on the specialist side and differ on the skills side — 18 agents and 47 skills against 18 profiles and 52 skills — and Section 4 explains why that difference is expected rather than a sign that something is missing. Counts drift as the toolkits grow, and a distributed copy may carry fewer items than the tree they were counted from, so recount from the tree in front of you before citing a number.

3. Why the platform split shapes everything

Here is the mechanism that generates almost every difference you will see between the two toolkits. It is worth walking slowly, because once you see it operate on one feature you can predict how it operates on the rest.

Claude Code has deterministic hooks that fire on real events and can block. Claude Science has none of that — only a soft reviewer that reads after the fact. Consider what each platform must do to enforce the same discipline: *verify a claim against the record before a write lands*. On Claude Code, a hook (`claim-verify-guard.sh`) fires *before* the write, checks the claim, and if it does not match, exits with code 2 and the write never happens. That is a hard gate. On Claude Science there is no “before the write” event to hook into, so the same discipline can only be *approximated*: the profile runs a prose self-check, emits a marker into its output, and the background reviewer scores that marker afterward. That is a soft gate. Same intent; a hard mechanism on one side, the strongest available soft mechanism on the other.

Now watch the same logic produce a completely different-looking divergence. **The completion beep.** When a long task finishes, both toolkits want to alert the user. Claude Code fires a beep from a Stop-event hook (`xbeep`) using the local machine’s audio device — wired once, no per-agent code. Claude Science has neither a turn-end event nor an audio device, so it produces the sound *in the browser* through Web Audio, and — because there is no hook to fire it automatically — **every profile that wants the beep must declare it as a standing behavior.** The result is visible in the data: the `audible-alert` skill is referenced by 17 of the 18 CSRTB profiles (counted 7 August 2026; the exception is `SOFTWARE_DEVELOPER`, the newest profile). That near-total fan-in is not redundancy or sloppiness; it is the exact mechanical cost of a missing hook surface — and the single profile that does not carry it shows the other half of that cost, since a behavior every unit must re-declare is a behavior a new unit can silently miss. The same absence also reshapes sub-agent orchestration (Claude Code’s synchronous, shared-workspace Task tool versus Science’s asynchronous `host.delegate` with its collect-and-steer SDK) and provenance-tracking (a rule-plus-hook on Code, a metadata-database-reading skill on Science).

The general pattern — and this is the thing to carry forward — is: **when a platform lacks a shared interception point, a cross-cutting concern that was wired once as a hook must instead be re-declared in every unit that wants it.** You will meet this exact shape again outside these toolkits: a logging concern is middleware you write once in a framework that supports it, and a line repeated in every request handler in one that does not. The toolkits are one instance of a general fact about where a platform lets you centralize behavior.

4. The three tiers — the model that tells you what is shared

Because the two toolkits share a methodology but diverge by platform, every item in them falls into exactly one of three relationships. This is the single most useful thing to internalize, because it converts “why is this here and not there?” from a puzzle into a lookup. The authority for the classification lives in two machine docs — `SHARED_VS_DIVERGENT.machine.md` and `PORT_DIRECTION_EXCLUSIONS.machine.md` — and the tiers are these:

1. **Tier S — Shared exactly.** Same discipline, same *kind* of carrier (a skill maps to a skill, an agent to a profile), on both sides. These are the platform-neutral methodology items — the statistics skills, the domain-science skills, the writing skills, most of the modeling agents. A change to one *should* be mirrored to the other. **The crucial caveat:** “shared” means shared *name and discipline*, not shared *bytes*. Of the 40 same-name skills at the earlier measurement (not re-derived at today’s counts), **zero are byte-identical** across the two toolkits; the median body similarity is 0.96, and a few diverge much further (`machine.md` sits at 0.65). So even a Tier-S item carries platform-adapted wording inside — you mirror the *idea*, adjusting the platform vocabulary, never copy the file.
2. **Tier C — Shared conceptually, different mechanism.** The discipline is the same on both sides, but the *carrier* differs because the platforms differ — and here you must **never byte-copy across the boundary**. You re-express the concept in the other platform’s

mechanism. Every example from Section 3 is a Tier-C pair: verification (hooks vs. reviewer), the completion alert (xbeep vs. audible-alert), the cold-start handoff (baton vs. handoff-brief), sub-agent orchestration (Task tool vs. host.delegate), provenance (rule-plus-hook vs. provenance-guard skill), document rendering (folio vs. folio-science), and statistics guidance (a *skill* on Code, a *profile* on Science). A Tier-C item that copies the mechanism instead of re-expressing it is a defect — it drags one platform’s runtime into the other’s file.

3. **Tier P — Platform-only.** Meaningful on one platform, inapplicable on the other, and never ported. On the Science side: the project-local ECOSYSTEM_MODEL_TRACER profile. On the Code side: capability-audit and the entire xbeep hook family.

Why the roster counts differ, explained by tier. CCRT’s 18 agents and 47 skills against CSRTB’s 18 profiles and 52 skills is not an asymmetry to correct. The profile side carries the project-local ECOSYSTEM_MODEL_TRACER and a GENERALIST with no Code twin. And research-stats-advisor is a *skill* on Code but a *profile* on Science — a carrier difference, not a missing item. Meanwhile CCRT has agents Science lacks *as agents* (machine-doc-reviewer, sci-file-indexer, version-control-docs), some of which exist on the Science side under a different carrier, because a Science skill legitimately maps to a Code rule or hook. The rule when you meet any count difference: **classify the item into S, C, or P first.** An item you cannot classify defaults to “port candidate” (open work), never silently to “excluded.”

4.1 CCRT roster — 18 agents

Each agent lists the skills it names in its own file as collaborators; **bold** marks a skill it loads by default (always on), plain text marks one it loads on demand.

Specialist	What it does	Key skills it reaches for
agent-tooling-engineer	Builds and maintains Claude Code toolkit customizations with idempotent, portable installation mechanisms	bash-hook-contract , machine-md, research-stats-advisor, toolkit-extension-authoring
code-review-debugger	Code review, debugging, and optimization for R, Python, or MATLAB	julia-performance-correctness, mgcv-temporal-gam, temporal-block-cv, tz-safe-time-stamps
design-rationale-analyst	Recovers implicit rationale and design principles behind work, distinguishes transferable schema from instances	design-rationale, expert-prose-style, research-stats-advisor, teaching-narrative, writing-science
dynamical-systems-modeler	Builds simulation models of biological/ecological state evolution via ODE, PDE,	reproduce-model-from-literature, research-stats-advisor

Specialist	What it does	Key skills it reaches for
	matrix, agent-based, or biogeochemical approaches	
ecophysiology-modeler	Builds mechanistic plant and ecosystem models: photosynthesis, stomatal optimality, allocation trade-offs, temperature response	reproduce-model-from-literature, research-stats-advisor
formal-argument-checker	Verifies formal claims and quantitative reasoning through computation: deontic logic, base rates, signal detection, logical validity	(none — self-contained)
llm-doc-architect	LLM documentation, agent/skill design, and Claude Code migration to Claude Science	machine-md, plan, research-stats-advisor
machine-doc-reviewer	Reviews machine-facing docs against LLM best-practices: framing, triggers, examples, style, atom-preservation	machine-md
machine-learning-scientist	Physics-informed Bayesian ML for multi-source data fusion, gap-filling, and calibrated uncertainty quantification	calibrated-uq-for-ml, multi-source-fusion-bias-correction, research-stats-advisor, scientific-ml-fundamentals , temporal-block-cv , tree-ensembles
micromet-reconstructor	Fills gaps in tower microclimate data, interpolates drivers across heights, derives variables with domain physics	aggregation-jensen-bias, brms-hierarchical-fitting, gap-fill-imputation, mgcv-temporal-gam, micromet-height-interpolation, research-stats-advisor, temporal-block-cv, temporal-qc-outlier-detection, tz-safe-timestamps
ml-hybrid-process-modeler	Learns canopy flux profiles via physics-informed ML enforcing energy/mass conservation constraints	biosphere-atmosphere-flux-exchange , ml-emulator-surrogate, physics-informed-ml , research-stats-advisor

Specialist	What it does	Key skills it reaches for
planner	Decomposes tasks into routed, gated plans mapping subtasks to specialist agents and skills with execution topology	baton, collab, delegation-planning, directing-execution, eliciting-llm-behavior, machine-md, plan, preflight-parallel, request-archetypes , solo
prompt-engineer	Tighten draft prompts into concise, higher-efficacy rewrites with token reduction metrics	eliciting-llm-behavior , machine-md
research-data-manager	research-data organization, archival naming, keep-vs-discard decisions, provenance judgment across project lifecycles	research-stats-advisor
sci-file-indexer	Index scientific literature folders into metadata tables with DOI/CrossRef curation and duplicate detection	plan, sci-file-index
science-writing-stylist	Diagnoses and revises scientific prose structure using OCAR framework and sentence-level craft to improve clarity and reader comprehension	<i>(none — self-contained)</i>
software-developer	Implements, extends, and refactors research software to a spec — pipelines, modules, CLIs, and the tests that ship with them — and hands every substantial build to code-review-debugger instead of self-certifying	refactoring, software-craft , testing-discipline
version-control-docs	Manage code versions, create documentation, organize project structure, and preserve working code	plan

4.2 CSRTB roster — 18 profiles

Same convention. *Note:* every CSRTB profile except SOFTWARE_DEVELOPER also references audible-alert as a standing behavior (Section 3 explains why); it is omitted from the collaborator column below to keep the genuine working-sets visible.

Specialist	What it does	Key skills it reaches for
AGENT_TOOLING_ENGINEER	Engineering specialist for skills, profiles, kernel sidecars, delegation, toolkit hooks, settings management, and installer tiers	bash-hook-contract , machine-md, toolkit-extension-authoring
CODE_REVIEW_DEBUGGER	Expert review and debugging of scientific-computing code in R, Python, Julia, MATLAB with assertion validation	brms-hierarchical-fitting, julia-performance-correctness, mgcv-temporal-gam, temporal-block-cv, tz-safe-timestamps
DESIGN_RATIONALE_ANALYST	Recovers implicit rationale, design philosophy, and governing principles from any work	design-rationale, expert-prose-style, teaching-narrative, writing-science
DYNAMICAL_SYSTEMS_MODELER	Simulates biological and ecological state evolution using ODEs, PDEs, matrix models, agent-based approaches, and biogeochemical cycling	reproduce-model-from-literature
ECOPHYSIOLOGY_MODELER	Mechanistic plant and ecosystem function models: photosynthesis, stomatal optimization, carbon economics, allocation trade-offs	reproduce-model-from-literature
ECOSYSTEM_MODEL_TRACER	Traces mass/energy/water/carbon flow from solve variable through recorder to plotted output; validates S1 signatures	<i>(none — self-contained)</i>
FORMAL_ARGUMENT_CHECKER	Verifies formal claims, deontic logic validity, base-rate arithmetic, signal-detection theory, and hidden empirical premises	<i>(none — self-contained)</i>
GENERALIST	General-purpose research assistant with wide capability access and audible notifications	<i>(none — self-contained)</i>
LLM_DOC_ARCHITECT	LLM documentation, agent prompts, skill design, and Claude Code to Science porting	plan

Specialist	What it does	Key skills it reaches for
MACHINE_LEARNING_SCIENTIST	Physics-informed Bayesian ML for multi-source fusion, gap-filling, bias-correction, and calibrated uncertainty in forcing reconstruction	calibrated-uq-for-ml, multi-source-fusion-bias-correction, scientific-ml-fundamentals , temporal-block-cv , tree-ensembles
MICROMET_RECONSTRUCTOR	Fills microclimate data gaps and interpolates drivers vertically across tower heights with physics-based priors	aggregation-jensen-bias, brms-hierarchical-fitting, gap-fill-imputation , mgcv-temporal-gam, micromet-height-interpolation , temporal-block-cv, temporal-qc-outlier-detection, tz-safe-timestamps
ML_HYBRID_PROCESS_MODELER	Physics-informed ML for canopy microclimate modeling with energy-balance constraints and flux closure	biosphere-atmosphere-flux-exchange , ml-emulator-surrogate, physics-informed-ml
PLANNER	Decomposes tasks into routed, gated plans mapping subtasks to specialist profiles and execution topologies	collab, delegation-planning, eliciting-llm-behavior, handoff-brief, machine-md, plan, preflight-parallel, request-archetypes , solo
PROMPT_ENGINEER	Compresses verbose prompts into efficient, high-efficacy versions with diff and token metrics	eliciting-llm-behavior , machine-md
RESEARCH_DATA_MANAGER	Research-data lifecycle classification, organization, and provenance across long-running projects	<i>(none — self-contained)</i>
RESEARCH_STATS_ADVISOR	Research methodology and statistical guidance for complex hierarchical data analysis decisions	aggregation-jensen-bias, brms-hierarchical-fitting, mgcv-temporal-gam, temporal-block-cv
SCIENCE_WRITING_STYLIST	Diagnoses and revises scientific prose using OCAR story structure, paragraph, sentence, and word craft	writing-science

Specialist	What it does	Key skills it reaches for
SOFTWARE_DEVELOPER	Implements, extends, and refactors research software to a spec — pipelines, modules, CLIs, and their tests — and hands every substantial build to CODE_REVIEW_DEBUGGER instead of self-certifying	refactoring, software-craft, testing-discipline

4.3 CCRT skills — 47 as of 7 August 2026, grouped by function

The group tables below were compiled at an earlier count and list a representative selection rather than every skill; the per-group totals have not been re-derived against the current tree.

Orchestration (7 skills)

Skill	What it does
collab	Collaborative mode — surface non-trivial calls without gating every step, normal interactive default
delegation-planning	Map subtasks to specialist agents and skills; select cascade topology or single-thread execution
directing-execution	Direct multi-agent execution: supervise results, adapt plan, re-launch subagents as needed
plan	Map territory and surface go/no-go decisions before committing to scope-defining steps
preflight-parallel	Compute available CPU cores and launch independent runs in parallel while staying within headroom
request-archetypes	Identify request archetypes and route to appropriate specialist carriers and output forms
solo	Autonomous task execution with self-directed decisions, pausing only for user-only choices or fatal blockers

Methodology (*stats* / *ML* / *modeling-method* / *verification*) (15 skills)

Skill	What it does
aggregation-jensen-bias	Compute nonlinear quantities at native resolution before aggregating to avoid Jensen-inequality bias
brms-hierarchical-fitting	Fit hierarchical Bayesian models with temporal AR structure, latent effects, and chain diagnostics in brms
calibrated-uq-for-ml	Validate and repair ML predictive uncertainty intervals to match empirical coverage via split-conformal prediction
gap-fill-imputation	Impute autocorrelated time series gaps with brms/mgcv, chunk-predict with spliced overlaps, tier sources, verify fidelity
mgcv-temporal-gam	Fit temporal GAM with autocorrelated residuals, defensible smoothing basis selection, and skewed predictor handling
ml-emulator-surrogate	Build fast ML surrogates for expensive mechanistic simulators enabling large-scale parameter inversion and sensitivity analysis
multi-source-fusion-bias-correction	Harmonize gappy reference series with satellite and reanalysis data via bias correction over temporal overlap
physics-informed-ml	Physics-informed ML with hard and soft conservation constraints, gray-box closures, and PINN modes
reproduce-model-from-literature	Re-implement published models from equations, reproduce baseline results before extending
research-stats-advisor	Advises on choosing statistical methods, checking assumptions, and interpreting results for time-series, mixed-effects, GAM, Bayesian, and causal inference
scientific-ml-fundamentals	ML discipline layer for tall-forest flux-tower gap-fill, fusion, and reconstruction with calibrated uncertainty
temporal-block-cv	Temporal and blocked cross-validation for autocorrelated or imbalanced data with class-robust metrics
temporal-qc-outlier-detection	Detects spikes, drift, and level shifts in autocorrelated environmental time series with diurnal-cycle preservation
tree-ensembles	Gradient-boosted trees and random forests with temporal feature engineering and quantile objectives for environmental regression

Skill	What it does
tz-safe-timestamps	Construct and align timestamps across timezone-mismatched data sources with explicit verification

Domain-science (ecology / biophysics / amplicon) (7 skills)

Skill	What it does
biosphere-atmosphere-flux-exchange	Canopy turbulent transport, energy-balance closure, leaf-to-ecosystem flux scaling, eddy-covariance method
micromet-height-interpolation	Interpolates microclimate drivers (Tair, VPD, CO2, H2O, wind) from sparse tower heights to fine vertical grid for ecosystem models

Doc + provenance (authoring / rationale / LLM-form / literature / handoff) (11 skills)

Skill	What it does
baton	Author machine-record handoff document enabling cold-session resumption from document alone
design-rationale	Recover implicit design rationale and transferable principles from codebases, methods, datasets, or experimental programs
eliciting-llm-behavior	Catalog structural techniques for reliably eliciting specific model behaviors in prompts
expert-prose-style	Adopts expert flowing-prose register for domain-expert readers until user requests different style
folio	Translate machine-authored docs to human-readable PDF and docx with preflight tool setup and character verification
machine-md	Formats docs for LLM readers using trigger-conditioned framing, concrete examples, and terse machine style
scanned-pdf-ocr	Extract text from image-only or degraded PDFs using offline Tesseract OCR

Skill	What it does
sci-file-index	Build and maintain a searchable catalog of scientific papers with metadata extraction, filename resolution, OCR, and duplicate detection
sci-library-curate	Dedup scientific papers, migrate to clean folders by Author_Year_Journal_Title, classify into Topic/Subtopic tree with tags
teaching-narrative	Writes explanatory guides and tutorials grounded in worked examples for reader understanding and application
writing-science	Diagnose and revise science prose using OCAR story structures, given-to-new flow, and mechanical draft-level tells

Platform-ops (hooks / installer / compute / audio / migration) (6 skills)

Skill	What it does
bash-hook-contract	Bash/Python hook utilities: stdin-JSON parsing, exit-code mapping, portable timeout, CRT gating, atomic writes
capability-audit	Audit installed agents/skills for duplication and recommend retirement or relocation decisions
julia-performance-correctness	Diagnose Julia performance issues, type instability, dispatch problems, and correctness gotchas in numerical code
toolkit-extension-authoring	Author Claude Code customizations with idempotent installation, merge contracts, and verification gates

4.4 CSRTB skills — 52 as of 7 August 2026, grouped by function

Same caveat as Section 4.3: the groups below are a representative selection, not a re-derived enumeration.

Orchestration (8 skills)

Skill	What it does
collab	Balanced mode between autonomous solo work and deliberative planning; surfaces non-trivial calls without gating every step
delegation-planning	Map subtasks to specialist profiles and skills, select cascade topology or single-thread approach
directing-execution	Supervise multi-agent execution in flight, adapt plan mid-run, steer sub-agents, handle delegation runtime
plan	Map territory and surface plans for scope-defining steps before user approval
preflight-parallel	Measure available cores and RAM, then run multiple independent jobs in parallel without overloading
request-archetypes	Recognize request task archetypes and map to appropriate specialist carriers and output forms
solo	Runs tasks to completion autonomously, deciding all decidable questions without check-ins

Methodology (stats / ML / modeling-method / verification) (15 skills)

Skill	What it does
aggregation-jensen-bias	Aggregate nonlinear quantities by computing before averaging; preserve temporal and spatial marginals separately
brms-hierarchical-fitting	Fit hierarchical Bayesian models with temporal AR structure, custom latent effects, and chain diagnostics in brms
calibrated-uq-for-ml	Validate and repair ML predictive uncertainty calibration via empirical coverage testing and split-conformal widening
gap-fill-imputation	Impute autocorrelated time series with brms/mgcv, chunk-predict with overlapping splices, tier sources, verify continuity
mgcv-temporal-gam	Fit temporal GAM with autocorrelation, adaptive smoothing dimension, heavy-tail predictor handling

Skill	What it does
ml-emulator-surrogate	Build and validate fast ML surrogates for expensive mechanistic simulators enabling parameter inversion and sensitivity analysis
multi-source-fusion-bias-correction	Harmonize gappy reference series with satellite and reanalysis data via bias correction and fusion into one continuous record
physics-informed-ml	Physics-informed ML with hard or soft conservation constraints for hybrid models
reproduce-model-from-literature	Re-implement published models from equations, reproduce baseline results, then extend
scientific-ml-fundamentals	ML discipline layer for K67 forcing reconstruction: scope appropriately, benchmark against brms/mgcv, score on quality composite, ship calibrated uncertainty
temporal-block-cv	Temporal and blocked cross-validation folds for autocorrelated or imbalanced data with calibration-aware metrics
temporal-qc-outlier-detection	QC outlier detection for autocorrelated environmental time series with stratified diurnal preservation and layered degrading detectors
tree-ensembles	Gradient-boosted trees and random forests for tabular environmental regression with temporal feature engineering and quantile objectives
tz-safe-timestamps	Construct timezone-safe timestamps and explicitly verify alignment when joining or resampling multi-source temporal data
verification-loop	Verify computed state before writing durable claims; raises on mismatch or unchecked assertions

Domain-science (ecology / biophysics / amplicon) (8 skills)

Skill	What it does
biosphere-atmosphere-flux-exchange	Canopy turbulent transport, energy-balance closure, leaf-to-ecosystem flux scaling, eddy-covariance method

Skill	What it does
km67-canonical-methods	Registry of canonical gap-fill and height-interpolation methods for km67 Tapajos tower variables with lineage verification
micromet-height-interpolation	Interpolate microclimate drivers (Tair, VPD, CO2, H2O, wind) across height onto fine vertical grid from discrete tower sensors

Doc + provenance (authoring / rationale / LLM-form / literature / handoff) (15 skills)

Skill	What it does
design-rationale	Recover implicit design rationale, philosophy, and generalizable principles from work artifacts
doc-pipeline	Author or render machine-md documents with human prose translation and PDF output, gated for quality
durable-doc-architecture	Set up and audit durable reference documents with 20 categories, ownership rules, and status headers for cross-session agent orientation
eliciting-llm-behavior	Elicit specific model behaviors and reasoning paths using structural techniques
expert-prose-style	Expert prose style for domain readers; persists until changed
folio-science	Render Markdown/diagrams to PDF, docx, or image artifacts offline
handoff-brief	Write a cold-start brief and starter prompt for resuming this Claude Science project in a new conversation
machine-md	Writing or editing text for LLM readers—skills, prompts, memory, delegation briefs
provenance-guard	Detects files written to /tmp that feed model fits or published outputs, preventing provenance loss on kernel restart
provenance-over-description	Verify system facts from primary records, not descriptions
scanned-pdf-ocr	Extract text from scanned PDFs using Tesseract or vision model

Skill	What it does
sci-file-index	Catalog scientific-literature folders with confidence-tiered meta-data, resolve cryptic filenames to DOIs, flag duplicates and inconsistencies
sci-library-curate	Dedup scientific-literature library, migrate to canonical names, organize by topic hierarchy with tags
teaching-narrative	Writes explanatory documents with worked examples so readers understand and apply concepts
writing-science	Diagnose and revise science prose using Writing Science frameworks and mechanical draft-level detectors

Platform-ops (hooks / installer / compute / audio / migration) (5 skills)

Skill	What it does
audible-alert	Audible and visual in-browser alert when long Science kernel tasks finish, with 14 built-in synthesized sounds
bash-hook-contract	Bash/Python hook implementation with portable timeout, exit-code mapping, atomic writes, and CRT gating
julia-performance-correctness	Diagnose Julia performance allocations, type instability, dispatch and correctness issues in numerical code
toolkit-extension-authoring	Authoring Claude Code customizations that install idempotently and non-destructively via hooks, skills, slash commands, or settings fragments

5. The collaboration working-sets — which specialists and skills are designed to compose, and why

This is the section a roster alone cannot give you. Knowing that delegation-planning and directing-execution exist tells you nothing about *when to reach for them together*; that knowledge lives in how the planner is wired. What follows is the recovered rationale for each designed working-set — grounded in the actual `skills_referenced` edges each specialist declares in its own file, so these are not guesses about intent but readings of the wiring. Where a claim is my inference rather than something the files state, I say so; where a pattern holds only for these specific carriers rather than for any comparable design, I mark it as such.

5.1 The planner set — a router, not a doer

The `planner` (CCRT) / `PLANNER` (CSRTB) persona is the orchestration hub, and its wiring tells a precise story. It references a whole family of routing skills — `request-archetypes`, `delegation-planning`, `preflight-parallel`, and the three execution-mode skills `plan` / `solo` / `collab` — but it loads **only request-archetypes by default**. Everything else it names as an on-demand reference. Read that design directly: recognizing *what kind of request* you are facing is always-on, because it is cheap and it happens first; the heavier routing machinery — mapping each subtask to a specialist and picking a delegation topology — loads only once a plan is actually being built. A planner that eager-loaded every routing skill would pay that context cost on every trivial single-step request that never fans out. Splitting always-on recognition from on-demand routing keeps the persona cheap until a cascade is genuinely chosen. Both planner descriptions state the composition outright: each “loads the delegation-planning skill” and “satisfies its Delegation & Routing mandate by construction.” (*This router-not-doer pattern generalizes: CrewAI’s hierarchical-manager agent and the LangGraph supervisor pattern both instantiate a dedicated orchestrator that holds routing logic rather than doing the work. The specific load-by-default split, though, is particular to this harness.*)

The one place the planner splits by platform. This same shared persona contains the largest deliberate Code-versus-Science divergence in either toolkit, and it is instructive. The Code planner also references `directing-execution` and `baton`; the Science `PLANNER` references neither, substituting `handoff-brief`. The reason is exactly the Section-3 mechanism: the *design-time* vocabulary — the four topologies (parallel-wave, sequential-build, convergence, verify-loop), the request archetypes — is platform-neutral and fully shared, but the *run-time* supervise-loop is bound to each platform’s sub-agent SDK. Code’s Task tool runs a sub-agent to completion with no mid-run steering; Science’s `host.delegate` is asynchronous, with separate calls to collect results, send steering messages, and stop a child. One shared run-time skill would be impossible — it would have to name both SDKs. So `directing-execution` measures only 0.66 similar across the two toolkits, and `baton` / `handoff-brief` are a Tier-C renamed pair. The routing brain is shared; the hands are platform-specific.

5.2 The document-authoring cascade — separable stages, not one author

Authoring is deliberately built as a *chain of single-responsibility carriers* rather than one skill that both thinks and writes — and you are reading its output, because this document was produced by running that chain. The `design-rationale-analyst` / `DESIGN_RATIONALE_ANALYST` specialist references four authoring skills, and each does exactly one job:

- `design-rationale` **recovers** the governing logic of a body of work — the *thinking*. (It produced Section 5’s analysis of these very working-sets.)
- `teaching-narrative` **renders** that recovered content as a human explainer — the *writing*. Its own reference block names it “the render engine for the recovered content.” (It produced the prose you are reading now.)
- `writing-science` **revises** existing prose toward publication — compress, cut, funnel to the claim.

- expert-prose-style **sets the register** — a toggle for expert flowing prose, not a task in itself.

Why chain them instead of fusing them? Because a single author-everything skill would weld content-recovery to prose-style, and you would lose the ability to reuse a recovered rationale in a *machine* document, or to restyle prose without re-deriving its content. Splitting lets the analyst recover a rationale once and render it many ways — as this task did, producing first a terse machine reference and then this human twin from the same recovered content. (*The “separable carriers” principle generalizes well — the Diátaxis documentation framework similarly refuses one-doc-does-everything — but note it splits documents by reader **intent**, where this cascade splits an authoring task into **stages**. The stage split is specific to producing one artifact.*) Beneath all of them sits `machine-md`, the shared form primitive for any LLM-read text; it is referenced not only by the doc specialists (`llm-doc-architect`, `prompt-engineer`, the Code-side `machine-doc-reviewer`) but by both planners, because task briefs and profiles are themselves LLM-read. One layer below the personas sit the render backends `folio` / `folio-science`, which — tellingly — *no* agent or profile references directly; they are reached only through the render pipeline, kept decoupled so the same authored content can target Code’s local renderer or Science’s offline one without the author knowing which.

5.3 The modeling stacks — the physics owner defers method-choice to a statistics authority

The three scientific-modeling specialists share one discipline visible directly in their edges: **each owns its physics or mechanism but defers the choice of statistical method to a dedicated statistics authority**. Look at the wiring:

- `micromet-reconstructor` loads its domain skills (`micromet-height-interpolation`, `gap-fill-imputation`, `aggregation-jensen-bias`) and its QC skills, but references `research-stats-advisor` for method choice — and its description says so in plain words: it “defers statistical method choice to `research-stats-advisor`.”
- `ml-hybrid-process-modeler` loads `physics-informed-ml` and `biosphere-atmosphere-flux-exchange` by default (its mechanism), and references `research-stats-advisor` (its method authority).
- `machine-learning-scientist` loads `scientific-ml-fundamentals` and `temporal-block-cv` by default, and likewise refers method questions outward.

The statistics authority itself is a Tier-C carrier pair: a *skill* (`research-stats-advisor`) on Code, a *profile* (`RESEARCH_STATS_ADVISOR`) on Science. Why is this separation designed in? Because a modeler that also adjudicated its own cross-validation scheme, its own priors, its own hierarchical structure would be grading its own statistics — and the failure mode of a physics modeler judging its own fit is defending in-sample performance. Routing method-choice to a separate authority keeps the mechanism-owner from marking its own homework. (*This holds as a schema specifically where a dedicated statistics-methodology carrier is actually instantiated; in a fused applied-ML team where one engineer both builds the model and picks the split, the separation degrades to a role-hygiene preference rather than a structural guarantee.*) The three modelers also partition the flux problem cleanly among themselves — reconstruct the above-canopy boundary (`machine-learning-scientist`), map the above-to-within-canopy transfer through the flux mechanism (`ml-hybrid-process-modeler`), `gap-fill` and

vertically interpolate the drivers (`micromet-reconstructor`) — and their descriptions cross-reference each other’s boundaries explicitly so a task lands with the right one.

5.4 The verification chain — compute the claim, and keep the checker uncoupled

The quality-control specialists are wired for a specific kind of trustworthiness: they **compute** claims rather than reason around them, and they are deliberately low-coupling. `formal-argument-checker` references **zero** skills — it is entirely self-contained, because, as its description says, it verifies claims “by computing them, not by reading around them.” `code-review-debugger` references only the handful of method skills it actually validates against (`julia-performance-correctness`, `temporal-block-cv`, and the `timezone` and `GAM` skills), and the Science profile adds “validating assertions after each fix.” The design reason to keep a checker uncoupled is that a checker which cited *other* checkers could launder an unverified claim through a chain of references; a self-contained checker that recomputes the claim itself has no such escape hatch. *(This generalizes to compute-capable verification carriers specifically — an automated type-checker computes and does not defer, fitting the pattern, whereas a human reviewer who reads around it does not — so the schema is bounded to the class of checkers that can actually recompute the thing they check.)* One tier up from the individual checkers sits the platform-set hardness of the whole verification gate, the Tier-C boundary from Section 3: Code blocks with a hook, Science scores with a reviewer, and the Code form is the stronger of the two.

6. How to use all of this

Put the pieces together and the toolkits become directly actionable:

- **Starting a multi-part task?** Reach for the planner persona. It will recognize the request archetype first, then — only if the work genuinely fans out — pull in `delegation-planning` to route subtasks and pick a topology, and `preflight-parallel` to size the concurrency. On Code the run-time supervision is `directing-execution`; on Science it is the delegate SDK. Do not hand-assemble these; the planner composes them by construction.
- **Producing a document?** Decide which stage you are at. Recovering the logic of a body of work is `design-rationale`; teaching a human is `teaching-narrative`; polishing a manuscript toward publication is `writing-science`; writing for an LLM reader is `machine-md`. They compose — recover once, render many ways.
- **Building a model?** Pick the modeler that owns your mechanism (state-evolution, optimality, flux-transfer, or reconstruction), let it own the physics, and let it defer the statistics to the stats authority rather than deciding the method itself.
- **Porting a feature between the toolkits?** Classify it into a tier *first*. Tier S: mirror the idea, adjusting platform vocabulary — never copy the bytes. Tier C: re-express the discipline in the other platform’s mechanism — never drag a `host.*` call into a Code hook or vice versa. Tier P: do not port it at all. When in doubt, treat it as a port candidate and open the question, never as a silent exclusion.

The reward for learning the one platform difference and the three-tier model is that the roster stops being a wall of 135 near-duplicate names and becomes a map you can navigate: you know which specialist to summon, which skills it will bring, and exactly what will and will not survive a crossing between the two platforms.

7. Side-by-side comparison (code-built, verbatim)

Side-by-side comparison — CCRT (Claude Code) vs CSRTB (Claude Science)

Tier legend: **S** = shared role/skill (same name + discipline; system-prompt/skill bodies carry platform-adapted wording) · **C** = shared discipline via a DIFFERENT carrier (never byte-copy) · **P** = platform-only (never ported). Authority: SHARED_VS_DIVERGENT.machine.md, PORT_DIRECTION_EXCLUSIONS.machine.md.

Measured note (earlier-roster measurement; not re-derived at today's counts): of the 35 same-name shared skills then paired, 0 are byte-identical across platforms; median body similarity is 0.95 (20 cosmetic ≥ 0.90 , 7 light-adaptation, 8 substantive < 0.70). "Shared" means shared name + discipline, not shared bytes.

Roles — CCRT agents (18) ↔ CSRTB profiles (18)

CCRT agent	CSRTB profile	Tier	Relationship
agent-tooling-engineer	AGENT_TOOLING_ENGINEER	S	shared role, same charter
code-review-debugger	CODE_REVIEW_DEBUGGER	S	shared role, same charter
design-rationale-analyst	DESIGN_RATIONALE_ANALYST	S	shared role, same charter
dynamical-systems-modeler	DYNAMICAL_SYSTEMS_MODELER	S	shared role, same charter
ecophysiology-modeler	ECOPHYSIOLOGY_MODELER	S	shared role, same charter
formal-argument-checker	FORMAL_ARGUMENT_CHECKER	S	shared role, same charter
llm-doc-architect	LLM_DOC_ARCHITECT	S	shared role, same charter
machine-learning-scientist	MACHINE_LEARNING_SCIENTIST	S	shared role, same charter
micromet-reconstructor	MICROMET_RECONSTRUCTOR	S	shared role, same charter
ml-hybrid-process-modeler	ML_HYBRID_PROCESS_MODELER	S	shared role, same charter
planner	PLANNER	S	shared role, same charter
prompt-engineer	PROMPT_ENGINEER	S	shared role, same charter

CCRT agent	CSRTB profile	Tier	Relationship
research-data-manager	RESEARCH_DATA_MANAGER	S	shared role, same charter
science-writing-stylist	SCIENCE_WRITING_STYLIST	S	shared role, same charter
software-developer	SOFTWARE_DEVELOPER	S	shared role, same charter
research-stats-advisor (<i>SKILL</i>)	RESEARCH_STATS_ADVISOR (<i>profile</i>)	C	carrier asymmetry: CC = skill, CS = profile
machine-doc-reviewer	—	P	CCRT-only agent (CS: machine- md discipline via skill/reviewer)
sci-file-indexer	—	P	CCRT-only agent (CS: sci-file- index SKILL, no indexer profile)
version-control-docs	—	P	CCRT-only agent (Science has no local git)
—	ECOSYSTEM_MODEL_TRACER	P	P — project-local (CliMA/Emer- ald)
—	GENERALIST	—	CSRTB-only profile (no CC agent twin)

Skills — CCRT (47) ↔ CSRTB (52)

CCRT skill	CSRTB skill	Tier	Relationship
aggregation-jensen-bias	aggregation-jensen-bias	S	shared skill (name + discipline; platform-adapted body)
bash-hook-contract	bash-hook-contract	S	shared skill (name + discipline; platform-adapted body)
biosphere-atmosphere- flux-exchange	biosphere-atmosphere- flux-exchange	S	shared skill (name + discipline; platform-adapted body)
brms-hierarchical-fitting	brms-hierarchical-fitting	S	shared skill (name + discipline; platform-adapted body)
calibrated-uq-for-ml	calibrated-uq-for-ml	S	shared skill (name + discipline; platform-adapted body)

CCRT skill	CSRTB skill	Tier	Relationship
collab	collab	S	shared skill (name + discipline; platform-adapted body)
delegation-planning	delegation-planning	S	shared skill (name + discipline; platform-adapted body)
design-rationale	design-rationale	S	shared skill (name + discipline; platform-adapted body)
directing-execution	directing-execution	S	shared skill (name + discipline; platform-adapted body)
eliciting-llm-behavior	eliciting-llm-behavior	S	shared skill (name + discipline; platform-adapted body)
expert-prose-style	expert-prose-style	S	shared skill (name + discipline; platform-adapted body)
gap-fill-imputation	gap-fill-imputation	S	shared skill (name + discipline; platform-adapted body)
julia-performance-correctness	julia-performance-correctness	S	shared skill (name + discipline; platform-adapted body)
machine-md	machine-md	S	shared skill (name + discipline; platform-adapted body)
mgcv-temporal-gam	mgcv-temporal-gam	S	shared skill (name + discipline; platform-adapted body)
micromet-height-interpolation	micromet-height-interpolation	S	shared skill (name + discipline; platform-adapted body)
ml-emulator-surrogate	ml-emulator-surrogate	S	shared skill (name + discipline; platform-adapted body)
multi-source-fusion-bias-correction	multi-source-fusion-bias-correction	S	shared skill (name + discipline; platform-adapted body)
physics-informed-ml	physics-informed-ml	S	shared skill (name + discipline; platform-adapted body)

CCRT skill	CSRTB skill	Tier	Relationship
plan	plan	S	shared skill (name + discipline; platform-adapted body)
preflight-parallel	preflight-parallel	S	shared skill (name + discipline; platform-adapted body)
reproduce-model-from-literature	reproduce-model-from-literature	S	shared skill (name + discipline; platform-adapted body)
request-archetypes	request-archetypes	S	shared skill (name + discipline; platform-adapted body)
scanned-pdf-ocr	scanned-pdf-ocr	S	shared skill (name + discipline; platform-adapted body)
sci-file-index	sci-file-index	S	shared skill (name + discipline; platform-adapted body)
sci-library-curate	sci-library-curate	S	shared skill (name + discipline; platform-adapted body)
scientific-ml-fundamentals	scientific-ml-fundamentals	S	shared skill (name + discipline; platform-adapted body)
solo	solo	S	shared skill (name + discipline; platform-adapted body)
teaching-narrative	teaching-narrative	S	shared skill (name + discipline; platform-adapted body)
temporal-block-cv	temporal-block-cv	S	shared skill (name + discipline; platform-adapted body)
temporal-qc-outlier-detection	temporal-qc-outlier-detection	S	shared skill (name + discipline; platform-adapted body)
toolkit-extension-authoring	toolkit-extension-authoring	S	shared skill (name + discipline; platform-adapted body)
tree-ensembles	tree-ensembles	S	shared skill (name + discipline; platform-adapted body)

CCRT skill	CSRTB skill	Tier	Relationship
tz-safe-timestamps	tz-safe-timestamps	S	shared skill (name + discipline; platform-adapted body)
writing-science	writing-science	S	shared skill (name + discipline; platform-adapted body)
folio	folio-science	C	same discipline, platform- named (render backend)
baton	—	—	CCRT-only skill
capability-audit	—	—	CCRT-only skill
research-stats-advisor	—	—	CCRT-only skill
—	audible-alert	—	CSRTB-only skill
—	doc-pipeline	—	CSRTB-only skill
—	durable-doc-architecture	—	CSRTB-only skill
—	handoff-brief	—	CSRTB-only skill
—	km67-canonical-methods	—	CSRTB-only skill
—	provenance-guard	—	CSRTB-only skill
—	provenance-over-description	—	CSRTB-only skill
—	verification-loop	—	CSRTB-only skill